

# Pokemon-Datenbankprojekt

Planung, Datenmodell, SQL-Abfragen und Reflexion

## Fokus

Relationale Struktur statt  
Einzeltabelle

## Datenbasis

21 Karten, 29 Attacken-  
Verknuepfungen

## Ziel

Abfragen erklaren, Model-  
lentscheidungen begruenden

Team: Robert, Joel, Berkay, Felix, David

# Agenda

| Teil | Kapitel                | Leitfrage                                                                            |
|------|------------------------|--------------------------------------------------------------------------------------|
| 1    | Projektplanung         | Was war MUSS/SOLL/KANN und welche Entscheidung hielt das Projekt im Zeitrahmen?      |
| 2    | Datenmodell            | Welche Entitäten, Beziehungen, Schlüssel und Normalisierungsideen stecken im Modell? |
| 3    | SQL                    | Welche JOIN-, GROUP-BY- und HAVING-Abfragen zeigen, dass das Modell funktioniert?    |
| 4    | Qualität und Reflexion | Welche Tests, Lücken und Verbesserungen ergeben sich für eine zweite Version?        |

# Projektplanung: MUSS, SOLL, KANN

## MUSS

- Name
- Kartenummer
- Typ
- HP
- Attacken

Ohne diese Felder wäre die Datenbank fachlich nicht vorzeigbar.

## SOLL

- Illustrator
- Rarität
- Evolution

Diese Felder machen das Modell interessanter und zeigen echte Beziehungen.

## KANN

- Preis
- Schwäche
- Resistenz
- Rückzugskosten

Diese Daten sind sinnvoll, aber nicht entscheidend für den Prototyp.

# Projektplanung: wichtigste Entscheidung

**Erst Pflichtdaten stabil, dann Erweiterungen.**

## Warum das geholfen hat

- Die Datenbank war früh abfragbar.
- Fehler im Modell wurden vor der Abgabe sichtbar.
- Zusatzfelder konnten weggelassen werden, ohne das Kernziel zu gefährden.

## Konkrete Folge

- 21 Karten haben Name, Nummer, Typ und HP vollständig.
- Attacken sind nicht nur Text, sondern separat modelliert.
- Preis und Resistenz wurden als ehrliche Lücken dokumentiert.

# Datenmodellierung: Entitäten

| Entität     | Was wird gespeichert?               | Warum eigene Tabelle?                                   |
|-------------|-------------------------------------|---------------------------------------------------------|
| card        | Nummer, Name, HP, Verweise          | zentrale Entität, jede Auswertung startet hier          |
| type        | Feuer, Wasser, Kampf usw.           | Typnamen wiederholen sich bei vielen Karten             |
| rarity      | Common, Uncommon, Rare usw.         | verhindert Schreibvarianten und Redundanz               |
| illustrator | Name des Illustrators               | Namen sollen nicht mehrfach als Freitext stehen         |
| attack      | Name, Schaden, Beschreibung         | Attacken sind eigene Informationen                      |
| card_attack | Karte, Attacke, Reihenfolge, Kosten | technische Umsetzung der mehrwertigen Attackenbeziehung |

Warum nicht eine Tabelle? Typen, Raritäten, Illustratoren und Attackentexte würden wiederholt und Korrekturen wären fehleranfällig.

# Datenmodellierung: Tabellenübersicht statt Diagramm

## Kerntabellen

| Tabelle     | wichtige Felder                                     |
|-------------|-----------------------------------------------------|
| card        | id, card_number,<br>name, hp, FK-Felder             |
| attack      | id, name, damage,<br>description                    |
| card_attack | card_id, attack_id,<br>attack_order,<br>energy_cost |

## Nachschlagetabellen

| Tabelle          | Rolle im Modell                        |
|------------------|----------------------------------------|
| type             | wiederverwendbare<br>Typnamen          |
| rarity           | wiederverwendbare<br>Seltenheitsstufen |
| illustrator      | wiederverwendbare<br>Namen             |
| card auf<br>card | Evolution als<br>Selbstbezug           |

# Kardinalitäten: Beziehungen im Modell

| Beziehung               | Kardinalität | Bedeutung                                                | technische Umsetzung                                                           |
|-------------------------|--------------|----------------------------------------------------------|--------------------------------------------------------------------------------|
| Typ zu Karte            | 1:n          | Ein Typ kann bei vielen Karten vorkommen.                | <code>card.type_id</code> zu <code>type.id</code> .                            |
| Rarität zu Karte        | 1:n          | Eine Seltenheit gilt für viele Karten.                   | <code>card.rarity_id</code> zu <code>rarity.id</code> .                        |
| Karte zu Attacke        | n:m          | Eine Karte kann mehrere Attacken haben.                  | <code>card_attack</code> mit <code>card_id</code> und <code>attack_id</code> . |
| Karte zu Vorentwicklung | Selbstbezug  | Eine Karte kann sich aus einer anderen Karte entwickeln. | <code>evolves_from_card_id</code> zu <code>card.id</code> .                    |

# Normalisierung: Redundanz vermeiden

## Beispiel: Rarität und Typ

- Common, Uncommon, Rare stehen in `rarity`.
- Feuer, Wasser, Kampf stehen in `type`.
- In `card` steht nur die jeweilige ID.

## Problem ohne Normalisierung

- Tippfehler wie `Uncommen` würden eigene Gruppen erzeugen.
- Änderungen müssten in vielen Kartenzeilen korrigiert werden.
- Statistikabfragen würden unzuverlässig.
- Attacken als Spalten wären unflexibel bei mehr oder weniger Attacken.

# Schlüssel: Primärschlüssel und Fremdschlüssel

| Kategorie               | bei uns                                                | Funktion                                               |
|-------------------------|--------------------------------------------------------|--------------------------------------------------------|
| Primärschlüssel         | id in card, type, rarity, illustrator, attack          | identifiziert jeden Datensatz eindeutig                |
| logische Fremdschlüssel | z.B. card.type_id, card.rarity_id, card.illustrator_id | verbinden Karte mit Nachschlagetabellen                |
| Zwischenschlüssel       | card_attack.card_id und card_attack.attack_id          | verbinden Karten und Attacken                          |
| Selbstbezug             | card.evolves_from_card_id                              | bildet Evolutionsketten innerhalb derselben Tabelle ab |

Aktueller Stand: IDs sind vorhanden und inhaltlich konsistent. Echte FOREIGN KEY-Constraints fehlen noch.

# Datenqualität: Constraints und Wirkung

| Constraint  | aktueller Stand                           | welchen Fehler er verhindern würde                          |
|-------------|-------------------------------------------|-------------------------------------------------------------|
| PRIMARY KEY | vorhanden für zentrale Tabellen           | doppelte oder nicht eindeutig adressierbare Datensätze      |
| FOREIGN KEY | logisch genutzt, technisch nicht angelegt | Verweise auf nicht existierende Typen, Attacken oder Karten |
| NOT NULL    | kaum konsequent gesetzt                   | Karten ohne Name, Typ, HP oder Nummer                       |
| UNIQUE      | nicht konsequent gesetzt                  | doppelte Kartennummern oder doppelte Typnamen               |
| CHECK       | nicht genutzt                             | unplausible Werte wie negative HP oder leere Energiekosten  |

# SQL-Abfragen: roter Faden

**Jede Abfrage zeigt einen anderen Nutzen des Datenmodells.**

## **1 JOIN**

Kartenansicht  
aus mehreren  
Tabellen zusammensetzen.

## **2 GROUP BY**

Typen zählen  
und Durchschnittswerte  
bilden.

## **3 HAVING**

Gruppen nach  
berechneten  
Ergebnissen filtern.

## **4 SELF JOIN**

Evolutionen  
innerhalb der  
Kartentabelle  
auswerten.

# Abfrage 1: Gesamtansicht einer Karte

```
SELECT
  c.card_number,
  c.name,
  t.name AS type,
  c.hp,
  i.name AS illustrator,
  r.name AS rarity
FROM card c
JOIN type t ON t.id = c.type_id
JOIN illustrator i ON i.id = c.illustrator_id
JOIN rarity r ON r.id = c.rarity_id
ORDER BY c.id;
```

## Ergebnis

- 21 lesbare Kartenzeilen
- Pflichtdaten plus Typ, Illustrator und Rarität
- Beispiel: **Camerupt**, Feuer, 140 HP, Uncommon

Die ausgelagerten Tabellen lassen sich wieder zu einer kompakten Kartenansicht verbinden.

## Abfrage 1: Warum genau diese JOINS?

| Join-Bedingung                       | Bedeutung                                       | warum nicht anders?                                |
|--------------------------------------|-------------------------------------------------|----------------------------------------------------|
| <code>t.id = c.type_id</code>        | verbindet Karte mit kontrolliertem Typdatensatz | Textvergleich wäre anfällig für Schreibfehler      |
| <code>i.id = c.illustrator_id</code> | holt den Namen aus illustrator                  | Illustratorname müsste sonst in jeder Karte stehen |
| <code>r.id = c.rarity_id</code>      | holt die Seltenheitsstufe aus rarity            | Freitext-Raritäten würden Statistiken verfälschen  |

**Überleitung:** Wenn eine Einzelkarte sauber rekonstruierbar ist, kann man im nächsten Schritt alle Karten zusammen statistisch auswerten.

## Abfrage 2: Typverteilung mit GROUP BY

```
SELECT
  t.name AS type,
  COUNT(*) AS cards,
  ROUND(AVG(c.hp), 0) AS avg_hp
FROM card c
JOIN type t ON t.id = c.type_id
GROUP BY t.name
ORDER BY cards DESC, t.name;
```

### Ergebnis

| Typ        | Karten | Ø HP |
|------------|--------|------|
| Feuer      | 4      | 165  |
| Wasser     | 4      | 95   |
| Kampf      | 3      | 77   |
| Pflanze    | 3      | 80   |
| Psycho     | 3      | 97   |
| Elektro    | 2      | 140  |
| Finsternis | 2      | 70   |

## Abfrage 2: Aussage der Statistik

### Was gewinnt man daraus?

- Feuer und Wasser sind am häufigsten vertreten.
- Feuer hat im Datensatz den höchsten durchschnittlichen HP-Wert.
- Die Typ-Tabelle macht Gruppierung sauber, weil Typen nicht als Freitext verstreut sind.

### Überleitung

- GROUP BY fasst Datensätze zusammen.
- Danach kann man mit HAVING genau diese Gruppen filtern.
- Das ist passend für Fragen wie: Welche Karten haben genau zwei Attacken?

## Abfrage 3: Karten mit genau zwei Attacks

```
SELECT
  c.name,
  COUNT(*) AS attacks,
  c.hp
FROM card c
JOIN card_attack ca ON ca.card_id = c.id
GROUP BY c.id
HAVING COUNT(*) = 2
ORDER BY c.hp DESC, c.name;
```

### Ergebnis: 8 Treffer

|              |                   |
|--------------|-------------------|
| Ns Gelatwino | Ns Gelatropo      |
| Camerupt     | Erikas Ultrigaria |
| Team Rockets | Banette           |
| Kokowei      |                   |
| Team Rockets | Ns Flampion       |
| Digdri       |                   |

HAVING filtert erst nach dem Zählen.

Genau dafür braucht man die Gruppierung pro Karte.

## Abfrage 3: Warum das Modell hier hilft

### Die Zwischentabelle macht Attacken flexibel.

#### Mit card\_attack

- beliebig viele Attacken pro Karte möglich
- Zählen mit COUNT(\*)
- Filtern mit HAVING

#### Ohne Zwischentabelle

- feste Spalten wie attack1, attack2
- schwieriger bei drei oder mehr Attacken
- schlechtere Statistikabfragen

## Abfrage 4: Evolutionen per Self Join

```
SELECT
  c.name AS card,
  e.name AS evolves_from
FROM card c
LEFT JOIN card e
  ON e.id = c.evokes_from_card_id
WHERE c.evokes_from_card_id IS NOT NULL
ORDER BY c.id;
```

### Ergebnis

| Karte     | entwickelt aus |
|-----------|----------------|
| Digdri    | Digda          |
| Camerupt  | Camaub         |
| Gelatropo | Gelatini       |
| Gelatwino | Gelatropo      |
| Kokowei   | Owei           |

## Abfrage 4: Aussage und Überleitung

### Aussage

- 5 echte Evolutions-Links sind verknüpft.
- Die Tabelle card wird mit sich selbst verbunden.
- Dadurch braucht man keine eigene Evolutionstabelle für diesen einfachen Fall.

### Grenze der Daten

- Im Rohdatensatz gibt es 9 Entwicklungskarten.
- Nur 5 konnten sauber auf vorhandene Karten zeigen.
- Das leitet direkt zu Datenqualität und Tests über.

# Testen und Debugging

## Smoke Tests

- Anzahl der Karten: 21
- Anzahl der Attacken-Verknüpfungen: 29
- Pflichtfelder leer? Nein
- doppelte Kartennummern? Nein

## Beispiel-Prüfabfragen

```
SELECT COUNT(*) FROM card;  
SELECT COUNT(*) FROM card_attack;  
SELECT card_number  
FROM card  
GROUP BY card_number  
HAVING COUNT(*) > 1;  
PRAGMA foreign_key_list(card);
```

Ergebnis: Die Inhalte passen logisch zusammen, aber PRAGMA foreign\_key\_list zeigt keine technisch angelegten Fremdschlüssel.

# Datenlücken: Was fehlt für eine bessere DB?

| Datenbereich  | Stand                                     | Bedeutung                                                    |
|---------------|-------------------------------------------|--------------------------------------------------------------|
| Pflichtfelder | vollständig                               | Name, Nummer, Typ und HP sind bei allen 21 Karten vorhanden. |
| Resistenz     | 1 von 21                                  | fachlich fast leer, deshalb nur vorsichtig auswertbar        |
| Preis         | 0 von 21                                  | als KANN-Anforderung nicht umgesetzt                         |
| Evolutionen   | 5 echte Links<br>bei 9 Entwicklungskarten | mehrere Vorstufen fehlen in der Auswahl                      |
| Constraints   | teilweise nur logisch                     | Datenbank verhindert falsche Eingaben<br>noch nicht selbst   |

# Softwareentwicklung und Reflexion

## Was würden wir ändern?

- echte FOREIGN KEY-Constraints im Schema
- NOT NULL, UNIQUE, CHECK konsequent setzen
- card\_new entfernen oder sinnvoll integrieren
- Importskript statt manueller Einpflege

## Nächster sinnvoller Schritt

- Set-Tabelle mit Setname und Setnummer
- Preis als eigenes Feld oder Verlaufstabelle
- vollständige Evolutionsketten importieren
- kleine Such- oder Filteroberfläche für Karten

# Fazit

## **Die Grundstruktur funktioniert.**

Pflichtdaten sind vollständig, zentrale Beziehungen sind nachvollziehbar modelliert und SQL-Abfragen zeigen die Vorteile der Aufteilung.

## **Die zweite Version wäre technischer.**

Mehr Constraints, automatischer Import, bessere Zusatzdaten und vollständige Evolutionsketten.

## Fragen?